

手摸手之注册用户如何防止缓存穿透？

什么是缓存穿透

缓存穿透是指在使用缓存系统时，恶意或频繁地请求一个不存在于缓存中的数据，导致每次请求都需要查询数据库或其他数据存储系统，从而绕过了缓存的效果，严重影响系统性能。

这种情况通常发生在恶意攻击、大量请求缓存中不存在的数据或缓存数据过期后的高并发访问。

缓存穿透会导致以下问题：

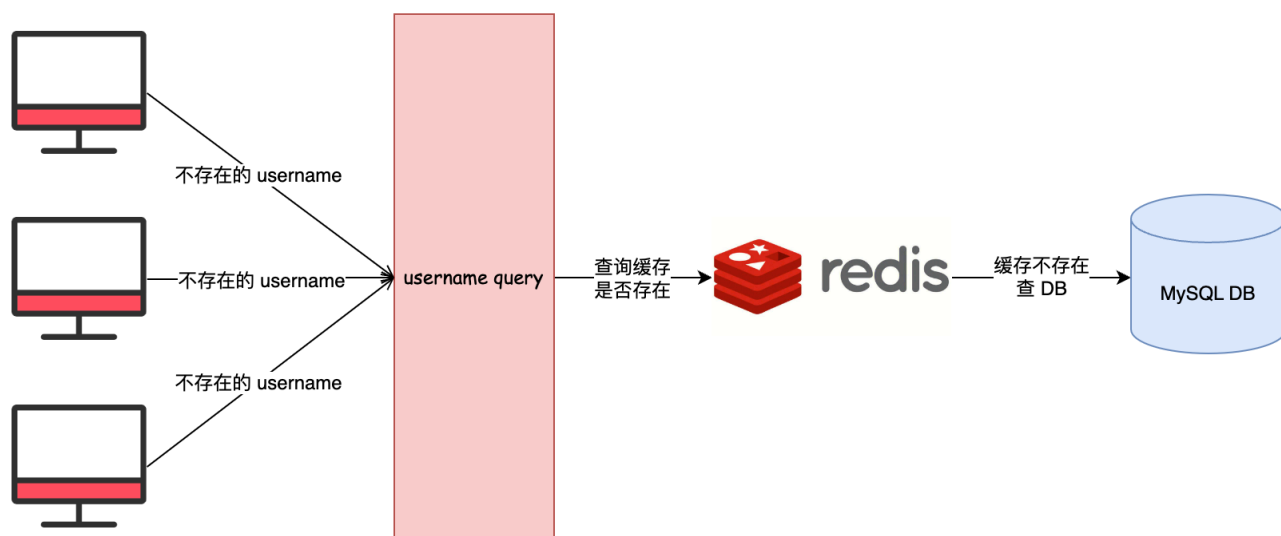
1. 频繁的查询数据库或其他数据存储系统，增加了数据库负载，降低了系统的吞吐量。
2. 大量的缓存不存在的数据请求可能会导致缓存服务器的内存被耗尽，影响其他正常的缓存操作。
3. 用户体验下降，因为请求的数据无法从缓存中获取，导致响应时间延长。

用户注册穿透场景

在高并发的会员注册场景下，可能会出现缓存穿透问题。主要原因可能是：

- 用户注册时，需要验证用户名是否已存在，这通常需要查询数据库。
- 如果缓存中没有该用户名，就会去数据库查询，如果数据库中也没有，就可以判断该用户名可用。
- 在高并发的情况下，可能有大量的新用户同时注册，输入的用户名极有可能都不存在于数据库中。这将导致大量的缓存不存在，都去查询数据库，造成数据库压力剧增。
- 且这些查询数据库的 Key 都不会被缓存，因为数据库中也没有，不会写入缓存。那么这些 Key 对应的 Null 值也不会被缓存，造成每次请求都查不到缓存，直接查询数据库。
- 这样就形成了缓存穿透情况。

而且极端情况下，注册的流程可能被恶意请求访问。注册请求缓存穿透流程图如下：



常见解决方案

所以，在用户注册场景下，需要注意防止缓存穿透，常见的处理方式有下述这些：

1. 对不存在的 Key 进行缓存，值设为 Null，并设置短暂过期时间，如 60 秒。
2. 使用布隆过滤器，将所有已注册的用户名存入布隆过滤器，判断时先判断该用户名是否在布隆过滤器中，不在的一定不存在，避免直接查询数据库。
3. 使用确定的数据结构如 Redis 的 Set 集合来存储已注册用户名，判断时检查是否在集合内。
4. 针对高并发注册场景，可以先查询缓存，如果不命中则使用分布式锁来保证只有一个线程访问数据库，避免重复查询。

但是，从真实业务场景来看，上面这些解决方案都存在弊端，不能适用于真实场景。

接下来我带大家一一解析，这些解决思路到底为什么不能用。

1. 不存在的 Key 进行缓存值设为 Null

对不存在的 Key 进行缓存，值设为 Null，并设置短暂过期时间，如 60 秒。

- 假设用户 A 注册 username 为 magestack，查询 DB 不存在，返回请求成功，并放入 Redis 缓存。但是用户 A 并没有使用该值作为 username。
- 如果有另一个用户 B 注册 username 为 magestack，将返回失败。也就是说每尝试一次不存在的用户名，该值 60 秒内都不可被注册。

结论：对用户使用体验不友好。此外，如果有大量并发请求查询不存在的用户名，可能会导致数据库短时间内被打挂。

2. 布隆过滤器

使用布隆过滤器，将所有已注册的用户名存入布隆过滤器，判断时先判断该用户名是否在布隆过滤器中，不在的一定不存在，避免直接查询数据库。

- 这种解决方案算是网上八股说的比较多的一个版本。但是依然不能解决实际场景问题。
- 如果用户注销了账号，该用户名就可以再次被使用。然而，布隆过滤器由于无法删除元素，因此无法处理这种情况。

结论：布隆过滤器不能删除元素的限制，导致该方案无法正式使用生产。

3. Redis Set 存储已注册用户名

使用确定的数据结构如 Redis 的 Set 集合来存储已注册用户名，判断时检查是否在集合内。

- 永久存储十几亿的用户信息到 Redis 缓存中显然不太现实，因为这会占用大量的内存资源。
- 即使是临时存储，如果在缓存中查询不到数据，仍然无法避免查询数据库的场景。
- 此外，对于这么多的用户信息，是否应该将其存储在一个 Key 中呢？显然是不可行的。即使进行分片，也会增加系统的复杂度。

结论：由于该方案占用内存较多且复杂度较高，因此不适合实际应用。

4. 分布式锁

针对高并发注册场景，可以先查询缓存，如果不命中则使用分布式锁来保证只有一个线程访问数据库，避免重复查询。

- 相对于上述解决方案，该方案在一定程度上可以解决会员注册缓存穿透的问题。
- 但是，如果在用户注册高峰期，只有一个线程访问数据库，这可能会导致大量用户的注册请求缓慢或超时。

结论：这对用户的使用体验来说并不友好，因此我们不建议使用该方案。

12306 如何解决注册穿透

如果没有用户名注销后可重复使用的需求，布隆过滤器无疑是最好的解决方案。但是考虑到企业的需求多样化，我们在设计时需要做好全方面的准备。

设计时需要考虑一个问题，布隆过滤器删除不了。如果已经加进去的用户名，无疑是无法再次复用的。

为此，我们可以通过再加一层缓存来解决这个问题。

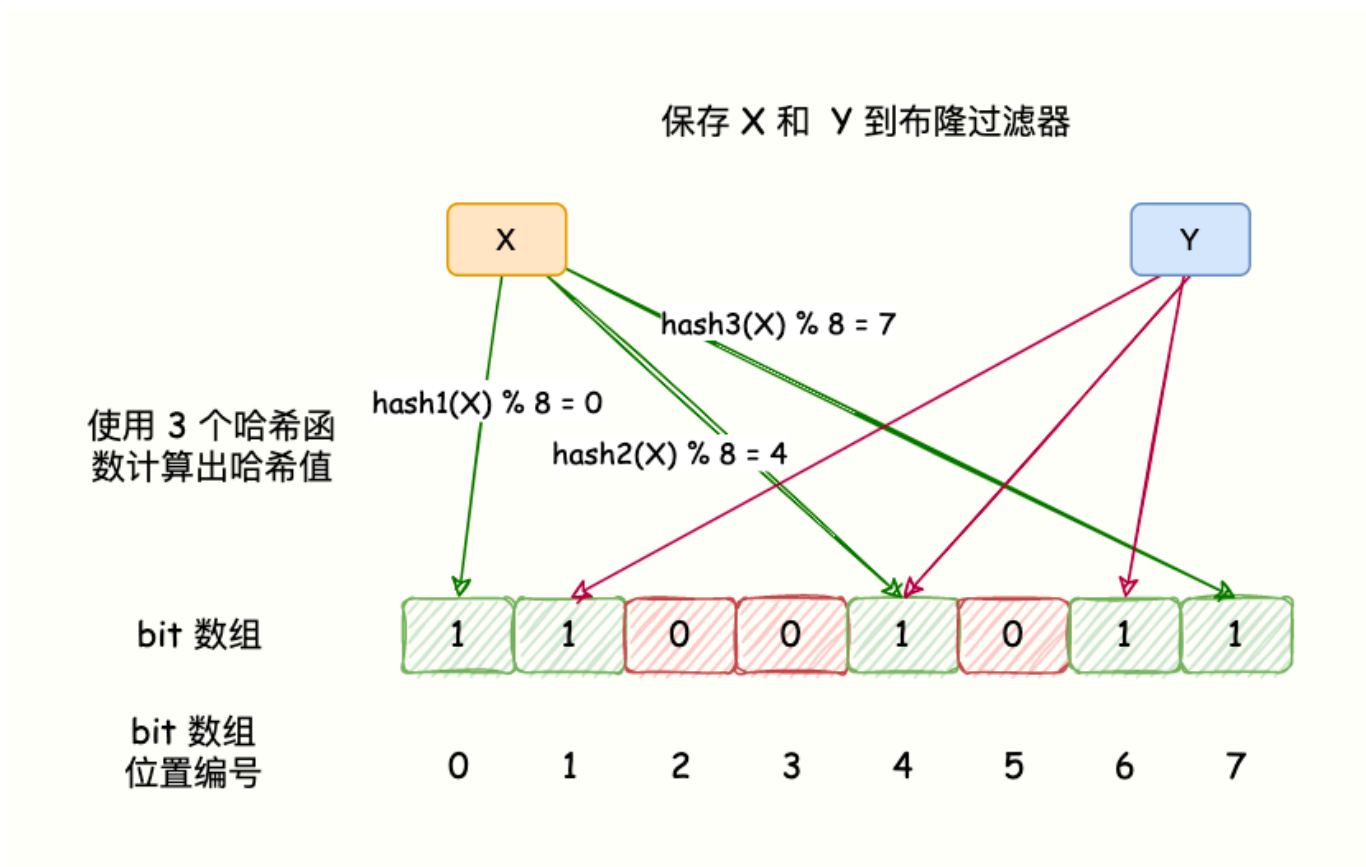
1. 什么是布隆过滤器

布隆过滤器 (Bloom Filter) 是一种高效的数据结构，用于判断一个元素是否存在于一个集合中。它利用位数组和多个哈希函数来实现快速的成员查询。

布隆过滤器的核心思想是用一个位数组（通常用二进制位表示）来表示一个集合，初始时所有位都置为0。

然后，对于每个要加入集合的元素，通过多个哈希函数计算出多个哈希值，然后将位数组中对应的位置置为1。

判断一个元素是否存在于集合时，同样使用多个哈希函数计算出对应的哈希值，然后检查位数组中对应的位置是否都为1，如果有任意一个位置为0，则说明该元素不存在于集合中；如果所有位置都为1，则该元素可能存在于集合中（因为有可能发生哈希碰撞），需要进一步查询底层数据结构来确认。



1) 关于布隆过滤器删除。

布隆过滤器在删除元素方面存在一定的限制，因为多个元素可能哈希到布隆过滤器的同一个位置。直接删除该位置的元素可能会影响其他元素的判断，因此布隆过滤器并不直接支持元素的删除操作。

2) 关于布隆过滤器查询误判。

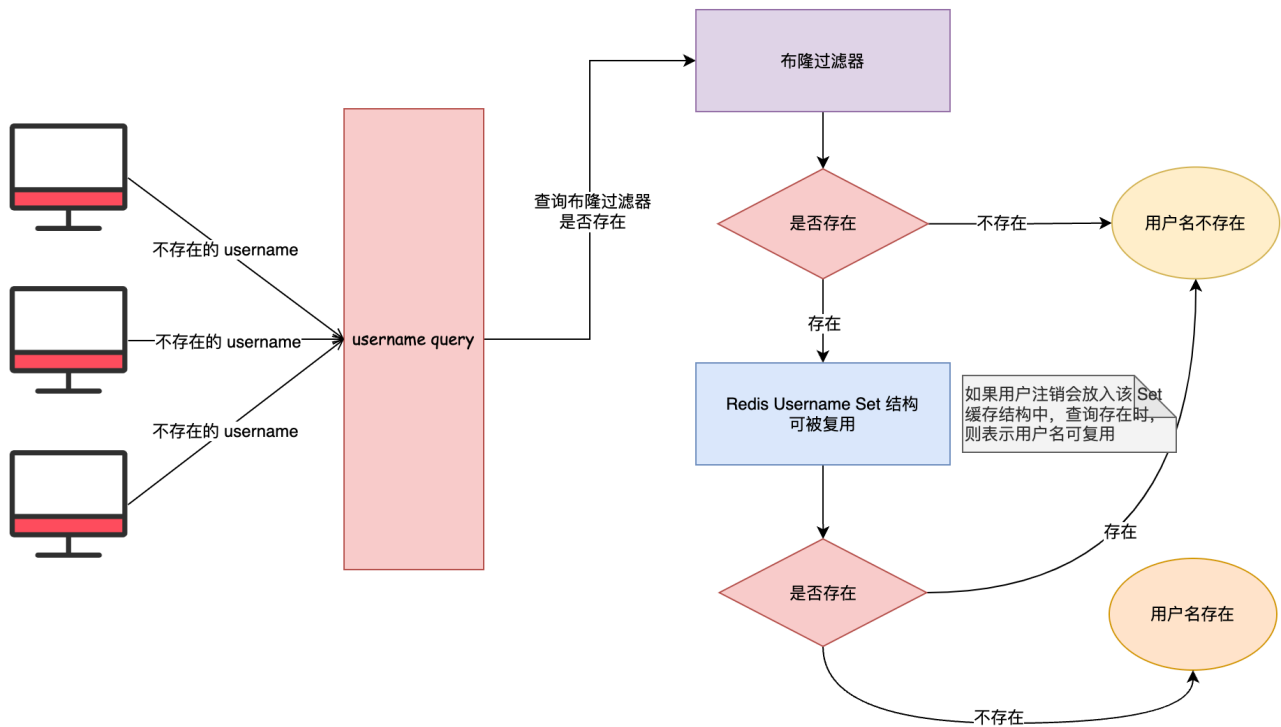
由于布隆过滤器使用多个哈希函数来映射元素到不同的位置，并且使用二进制位来表示元素的存在状态，这就导致了误判的可能性。具体来说，以下几个因素会导致误判：

1. 哈希碰撞：不同的元素经过哈希函数计算可能映射到相同的二进制位，导致不同元素在布隆过滤器中产生了冲突，从而可能被误判为存在于集合中。
2. 容量限制：布隆过滤器使用有限的二进制位数组来表示集合，当集合中元素数量较多时，可能会导致位的重复使用，从而增加误判的概率。

虽然查询元素是否存在有这误判率，但是如果查询元素是否不存在，则没有误判率。

2. 最终解决方案

上面说了，通过再加一层缓存来解决布隆过滤器无法删除的问题。具体流程如下图所示：



假设我们有一条用户名为 "mading" 的数据，注册后是如何不被重复注册，以及注销后又是如何能被再次使用的。

1. 用户名 "mading" 成功注册后，将其添加至布隆过滤器。
2. 当其他用户查询 "mading" 是否已被使用时，首先检查布隆过滤器是否包含该用户名。
3. 如果布隆过滤器中不存在该用户名，根据布隆过滤器的特点，可以确认该用户名一定没有被使用过，因此返回成功，证明该用户名可用。
4. 如果布隆过滤器中存在该用户名，进一步检查Redis Set结构中是否包含该用户名。如果存在，表示该用户名已被注销，同样可被再次使用。
5. 如果布隆过滤器中存在该用户名，但 Redis Set 结构中不存在，说明该用户名已被使用且尚未被注销，因此不可用。

1) Redis Set 结构是什么？

当用户注销后，系统会将其用户名放入缓存结构中。如果其他用户想要使用该用户名，会检查缓存是否存在该用户名。如果缓存中存在该用户名，就表示该用户名已被注销，可以被再次使用。

2) 使用这种方式后，会面临以下问题：

1. 查询性能消耗增加：由于采用了额外的 Redis Set 结构，查询过程需要进行两次查询，一次是查询布隆过滤器，另一次是查询 Redis Set 结构，这导致查询性能相比之前有所增加。
2. 存储损耗增加：相较于之前仅使用布隆过滤器的存储，现在需要额外存储 Redis Set 结构，这导致存储开销增加。

3) 上面这种方式真的是没有问题的么？

有一个问题可能会出现：如果用户频繁申请账号再注销，可能导致用户注销可复用的 Username Redis Set 结构变得庞大，增加了存储和查询的负担。

为了防止这种情况，我采取了以下解决方案：

1. 异常行为限制：每次用户注销时，记录用户的证件号，并限制证件号仅可用于注销五次。超过这个限制的次数，将禁止该证件号再次用于注册账号。
2. 缓存分片处理：对 Username Redis Set 结构进行分片。即使我们对异常行为进行了限制，如果有大量用户注销账户，存储这些数据在一个 Redis Set 结构中可能成为一个灾难，可能出现 Redis 大 Key 问题。因此，我将 Set 结构进行分片，根据用户名的 HashCode 进行取模操作，将数据分散存储在 1024 个 Set 结构中，从而有效地解决了这个问题。

4) [用户注册布隆过滤器容量设置以及碰撞率问题](#)

文末总结

缓存穿透是指在使用缓存系统时，恶意或频繁地请求一个不存在于缓存中的数据，导致每次请求都需要查询数据库或其他数据存储系统，严重影响系统性能。在高并发的用户注册场景下，可能会出现缓存穿透问题。

为了解决这个问题，我们可以采取多种解决方案，但每种方案都存在一些弊端。

- 对不存在的 Key 进行缓存值设为 Null：虽然可以避免重复查询数据库，但对用户体验不友好，且可能会导致数据库压力增加。
- 布隆过滤器：虽然可以快速判断是否存在于集合中，但无法处理删除元素的情况，且存在哈希碰撞导致误判的可能性。
- Redis Set 存储已注册用户名：占用内存较多且复杂度较高，不适合实际应用。

- 分布式锁：虽然可以解决并发查询数据库的问题，但影响用户注册的响应时间，不友好的用户体验。

最终，我们可以采用布隆过滤器结合缓存的方式来解决缓存穿透问题。通过布隆过滤器判断用户名是否可能存在，再通过缓存来确认是否真的存在，避免了对数据库的频繁查询。为了解决布隆过滤器无法删除的问题，我们再加一层缓存来存储已注销用户名，实现了用户名的可复用性。此外，为了防止缓存结构过大带来的问题，我们对缓存进行了分片处理，有效减轻了存储和查询的负担。

综上所述，综合考虑业务需求和系统性能，采用布隆过滤器结合缓存的解决方案是一个比较理想的解决方案，可以有效防止缓存穿透，并提升系统的性能和用户体验。

文中这些代码逻辑，可以查看以下方法：

1. org.opengoofy.index12306.biz.userservice.controller.UserInfoController#hasUsername
2. org.opengoofy.index12306.biz.userservice.controller.UserInfoController#register
3. org.opengoofy.index12306.biz.userservice.controller.UserInfoController#deletion

更新: 2023-09-12 21:28:55

原文: <<https://www.yuque.com/magestack/12306/go6vg8whk9g1lyhp>>